Playwright with TypeScript: compact cheatsheet

STEP 1 Getting started

1.1 What Playwright is and how it works

You write steps Open, type, click, check.

Messages Each call becomes a small message. The driver is the same for every language.

Real browsers Real clicks and real typing. The page cannot tell it apart from a person.

It waits Visible, stable, enabled. No fixed sleeps, so far fewer flaky tests.

Remember Playwright drives real browsers through one driver, waits for elements before it acts, and works the same way in Java, Python, JavaScript and TypeScript.

```
await page.goto('http://localhost:4173/login');
```
Open the sign in page.

```
await page.getByLabel('Email').fill('asha@shop.example');
```
Type the email into the box labelled Email.

```
await page.getByLabel('Password').fill('s3cret');
```
Type the password.

```
await page.getByRole('button', { name: 'Sign in' }).click();
```
Click the button called Sign in. The shop signs Asha in and opens the products page.

```
await expect(page.getByText('Hello, Asha')).toBeVisible();
```
Check that the greeting says Hello, Asha. The check waits until it appears.

TRAP People often ask for Playwright in C, C++ or Go. There is no official Playwright for C or C++. For Go there is a community project, playwright-go, which is not made by the Playwright team. This subject sticks to the official languages so that every example is supported and up to date.

1.2 Installing Playwright and your first project

Runtime Install your language first. Check it with a version command.

Package Pin the exact version.

Browsers Playwright brings its own. --with-deps adds the Linux libraries.

Record Click around, get code. Then clean it up and add a check.

Remember Install the runtime, the Playwright package and its browsers, record a first draft with codegen, then turn it into a real test with a clear name and a check.

TRAP Keep one Playwright version per project and update on purpose. Each Playwright release expects its own browser builds. After you update the package, run the install command again, or the tests stop with an error that says the browser executable does not exist.

STEP 2 Browser handling

2.1 Browsers, contexts, pages and devices

Browser Start once, about 0.5 s. Shared by many tests.

Context A fresh, empty profile. Its own cookies, storage and permissions.

Page One tab. Tabs in a context share the login.

Emulation Size, locale, time zone, location. Set on the context, nothing on your computer changes.

Remember Start one browser, give every test a new context, and set what the site should see, such as a phone or a German user, with context options.

```
headless: true,
```
no window. Use false to watch it work

```
slowMo: 100,
```
wait 100 ms before every action

```
geolocation: { latitude: 28.6139, longitude: 77.209 },
```
New Delhi

TRAP Emulation catches most layout and touch bugs and costs nothing. Some bugs only show on real hardware, such as a keyboard covering a field or a slow phone. Many teams run emulated phones on every commit and real devices in a cloud service before a release.

2.2 Navigation, frames, dialogs, files and events

Navigation goto waits for load. After a click, wait for the new URL.

Iframes Step in first. Then locate as usual.

Dialogs Dismissed by default. Handle them before the click.

Files Wait for downloads. Fill file inputs directly.

Remember Start waiting before the click for anything the click creates: a new URL, a popup, a dialog or a download.

```
await page.waitForURL('**/products');
```
the app moves on by itself

```
await page.goto('http://localhost:4173/frames');
```
Open the checkout page. The card form is an iframe from the payment company.

```
await page.frameLocator('iframe[title="Payment form"]').getByLabel('Card number').fill('4111 1111 1111 1111');
```
Step into the iframe and type the card number.

```
await page.frameLocator('iframe[title="Payment form"]').getByLabel('Expiry').fill('12/29');
```
Type the expiry date.

```
await page.frameLocator('iframe[title="Payment form"]').getByRole('button', { name: 'Pay ₹219' }).click();
```
Click Pay inside the iframe.

```
await expect(page.getByRole('status')).toHaveText('Paid with card ending 1111');
```
Back in the main page, the shop shows the result.

```
await download.saveAs('reports/' + download.suggestedFilename());
```
keep a copy

```
await expect(page.getByRole('listitem')).toHaveCount(5);
```
wait for the list

```
const sum = await page.evaluate(([a, b]) => a + b, [20, 199]);
```
pass values in

TRAP Do not wait for the network to go quiet. A chat widget or analytics script can keep it busy forever. Wait for what the user is waiting for: a URL, a heading or a message. Every check with expect already waits.

TRAP Use evaluate to read and to prepare, not to click. A click done with JavaScript skips the waiting and the visibility checks that make Playwright tests reliable, and it can pass on a button a real user could never press.

STEP 3 Finding and using elements

3.1 Locators: finding elements

Role and name What the user sees. The first choice.

Forms Boxes by their label. Placeholder when there is none.

Narrow down Card first, then button. hasText, has, hasNot.

One element Actions need exactly one. The error lists them all.

Remember A locator is searched again every time you use it, and an action needs exactly one match.

```
const cards = page.getByRole('listitem');
```
nothing is searched yet

```
await page.getByText('Remember me').click();
```
clicking a label ticks its box

```
await expect(page.getByTestId(/^product-/)).toHaveCount(5);
```
all product cards

```
await expect(add).toHaveCount(4);
```
counting many is fine

```
const error = await add.click().catch(e => e);
```
acting on many is not

```
await page.goto('http://localhost:4173/orders');
```
Open the orders page. It shows 5 of 7 orders per page.

```
await page.getByRole('button', { name: 'Total' }).click();
```
Click the Total column header to sort by total, smallest first.

```
await page.getByRole('row').filter({ hasText: 'John' }).getByRole('button', { name: 'Delete' }).click();
```
Find the row that mentions John, then the Delete button inside that row only.

```
await expect(page.getByRole('status')).toHaveText('Order #1004 deleted');
```
The status message confirms which order went away.

```
await page.getByRole('button', { name: 'Next' }).click();
```
Go to the next page of the table.

```
await expect(page.getByText('Page 2 of 2')).toBeVisible();
```
Check the page counter by its text.

```
const rows = page.getByRole('row').filter({ has: page.getByRole('cell') });
```
skip the header row

TRAP Long XPaths copied from the browser tools, such as /html/body/div[2]/div/form/input, break as soon as one box is added to the page. Recorded tests from older tools are full of them, and they are the first thing to replace when a team moves to Playwright.

TRAP first(), last() and nth() depend on the order of the page. When the order changes, they quietly pick another element. Prefer a filter by text when you can.

3.2 Actions and auto-waiting

Auto-waiting Checks before every action. No sleeps needed.

Timeouts The error says which check failed. Read it before using force.

Typing fill for most boxes. pressSequentially for key by key.

Forms check, selectOption, dragTo. Found by their labels.

Remember Never sleep. Every action waits until the element is ready, and when you need more, wait for the exact thing you need.

```
await help.hover();
```
the mouse rests on it

```
await page.mouse.move(0, 0);
```
move the mouse away

```
await help.focus();
```
keyboard users reach it with Tab

```
await search.fill('desk');
```
sets the whole value at once

```
await search.pressSequentially('mug', { delay: 50 });
```
one key at a time, like a person

```
await page.keyboard.press('Control+K');
```
a shortcut for the whole page

```
await page.goto('http://localhost:4173/forms');
```
Open the feedback form.

```
await page.getByLabel('Your name').fill('Asha');
```
Type the name into the box labelled Your name.

```
await page.getByLabel('Topic').selectOption('delivery');
```
Choose Delivery in the Topic drop down.

```
await page.getByLabel('Happy').check();
```
Pick the Happy radio button.

```
await page.getByLabel('Message').fill('The parcel came a day early.');
```
Write a short message.

```
await page.getByLabel('Send me the newsletter').check();
```
Tick the newsletter box.

```
await page.getByRole('button', { name: 'Send feedback' }).click();
```
Send the form. Playwright scrolls the button into view by itself.

```
await expect(page.getByRole('status')).toContainText('Topic: delivery, mood: happy');
```
The thank you line shows every value the page received.

```
await page.getByLabel('Topic').selectOption({ label: 'Payment' });
```
or by the text you see

```
await page.getByLabel('Order date').fill('2026-10-01');
```
dates use yyyy-mm-dd

```
await page.getByLabel('Volume').fill('8');
```
a slider

```
await done.hover();
```
a second move, so the page gets its dragover event

TRAP Reaching for force is often a sign the test found the wrong element, or that a real user could not click it either. Read the call log first.

3.3 Assertions that wait for the page

Web-first Retries until true. 5 seconds by default.

Text Exact or contains. Lists check order too.

Negative Waits for things to go. toHaveCount(0) for none.

Soft Keep going. Report every problem.

Remember Check the page with assertions that retry, and never read a value first and check it later.

```
const offer = page.getByTestId('offer');
```
shows up 1.2 s after the click

```
await expect(offer).toBeVisible();
```
retries until it shows

```
await page.goto('http://localhost:4173/products');
```
Open the products page.

```
await page.getByPlaceholder('Search products').fill('bottle');
```
Search for bottle.

```
await expect(page.getByRole('listitem')).toContainText('Water bottle');
```
One card is left, and it is the right one.

```
await expect(page.getByPlaceholder('Search products')).toHaveValue('bottle');
```
The search box keeps what was typed.

```
await page.getByPlaceholder('Search products').fill('phone');
```
Search for something the shop does not sell.

```
await expect(page.getByRole('button', { name: 'Add to cart' })).toHaveCount(0);
```
No Add to cart buttons are left. A negative check also waits.

```
await expect(page.getByText('Loading products…')).not.toBeVisible();
```
waits until it goes away

```
await expect.soft(page.getByTestId('product-1')).toContainText('₹25', { timeout: 1000 });
```
wrong on purpose

```
await expect.soft(page.getByTestId('product-4')).toContainText('Add to cart', { timeout: 1000 });
```
wrong on purpose

```
await expect.soft(page.getByTestId('product-5')).toContainText('₹349');
```
right

```
await expect(response).toBeOK();
```
a web-first check for API answers

```
expect(products).toHaveLength(5);
```
plain checks: no waiting, no await

```
expect(tries).toBeGreaterThanOrEqual(3);
```
the first two tries fail on purpose

```
const slowExpect = expect.configure({ timeout: 10_000 });
```
a copy of expect with other defaults

TRAP A rule that catches many flaky tests: never put await inside expect. Write `await expect(locator).toBeVisible()`, not `expect(await locator.isVisible()).toBe(true)`.

STEP 4 The test runner

4.1 The test runner: organise, tag, parameterise and configure

Runner Finds and runs tests. One per language.

Tags Run a slice. Smoke now, regression tonight.

Data One test, many rows. Each row is its own result.

Config Settings in one place. Projects for each device.

Remember Tests say what to check. The runner and its configuration decide where, how and with what settings they run.

```
test.fail();
```
known to fail today, so a failure is the expected result

```
timeout: 30_000,
```
one test may take 30 s

```
expect: { timeout: 5_000 },
```
one assertion may keep trying for 5 s

```
forbidOnly: !!process.env.CI,
```
a forgotten test.only fails the CI run

TRAP A test.only left in the code silently skips every other test. Set forbidOnly in the CI config so such a run fails instead.

4.2 Fixtures, hooks and set up once

Hooks Around every test. Simple and visible.

Fixtures Only when asked for. Set up, use, tear down.

Automatic Every test gets it. Guards and recordings.

Once per run Health check, data. Before any test.

Remember Put set up where it belongs: in the test if one test needs it, in a fixture if several do, and in a setup step if the whole run does.

```
await use(page);
```
the test runs here

```
{ name: 'setup', testMatch: /global\.setup\.[jt]s/ },
```
runs first

4.3 Parallel runs, retries, timeouts and sharding

Workers Tests at the same time. Each with its own browser.

Isolation No shared data. Lock what must be shared.

Retries Try again on failure. Flaky is still a bug.

Shards Split across machines. Merge the reports.

Remember Parallel runs only work when every test can run alone, in any order, next to any other test.

```
fullyParallel: true,
```
every test can run at the same time as any other

```
workers: 2,
```
two worker processes, each with its own browser

```
await expect(page.getByRole('row')).toHaveCount(6);
```
a header row and five orders

```
test('shows five products', async ({ page }) => {
```
no lock, runs alongside

```
await expect(page.getByTestId('offer')).toHaveText('Buy 2 mugs, get 1 free');
```
the 5 s default is enough

```
await expect(page.getByTestId('offer')).toBeVisible({ timeout: 1_000 });
```
1 s, but the offer needs 1.2 s

```
test.slow();
```
three times the test timeout: 90 s

```
await page.getByRole('button', { name: 'Pay now' }).click({ timeout: 3_000 });
```
this click only

TRAP Playwright Test also has a serial mode, test.describe.configure({ mode: 'serial' }), where the tests in a file run in order and the rest are skipped after a failure. The Playwright docs advise against it: isolated tests are faster and easier to retry.

STEP 5 Network and APIs

5.1 Network: watch, mock and change requests

Watch See every call. Wait for the one you need.

Mock Answer with your data. No server needed.

Change Real answer, one field changed. The rest stays real.

Replay Record once. Same data every run.

Remember Mock what you do not own or cannot control. Keep a few tests against the real API, so the mocks do not drift away from it.

```
const productsCall = page.waitForResponse('**/api/products');
```
start waiting before the page asks

```
const response = await route.fetch();
```
ask the real server

```
products.find((p: { name: string }) => p.name === 'Desk lamp').stock = 0;
```
then change one field

```
await page.routeFromHAR('hars/products.har', { url: '**/api/products' });
```
no update: answers come from the file

```
await page.routeWebSocket('**/ws/stock', ws => {
```
no real server is contacted

TRAP Replay matches the address and the method exactly, and for POST also the body. A request that matches nothing in the file is aborted, unless you pass notFound: fallback.

5.2 API testing and signing in once

API tests No browser at all. Status, data, errors.

Tokens Proof of who calls. Keep them in secrets.

Mix API for set up. Page for the check.

Sign in once Save the state. Every test starts signed in.

Remember Use the page for what the user sees. Use the API for everything you only need to get ready.

```
expect((await request.get('/api/cart')).status()).toBe(401);
```
nobody is signed in yet

```
await page.addInitScript((t: string) => localStorage.setItem('token', t), token);
```
the shop keeps its token here

```
await page.request.post('/api/cart', { data: { productId: 3 } });
```
arrange the cart without clicking

```
use: { ...devices['Desktop Chrome'], storageState: 'playwright/.auth/asha.json' },
```
every test starts signed in

```
await page.goto('/products');
```
no sign in steps here

```
await expect(page).toHaveURL(/\/cart$/);
```
not sent to /login

TRAP Keep real passwords and tokens out of test code. Read them from environment variables, such as process.env.SHOP_PASSWORD, os.environ["SHOP_PASSWORD"] or System.getenv("SHOP_PASSWORD"), and set them as secrets in CI. The framework chapter shows a .env file for local runs.

TRAP The state file holds a live session that anyone could use. Add playwright/.auth to .gitignore, and let each run create it fresh.

STEP 6 Debugging and quality

6.1 Debugging failed tests: errors, traces, reports and the Inspector

Error Expected and received. Read it first.

Evidence Screenshot, video, trace. Kept on failure.

Trace Every action replayed. DOM, network, console.

Inspector Run it yourself. One step at a time.

Remember Read the error, then the trace, and only then run the test by hand. Most failures are solved at the first step.

```
await expect(page.locator('#hello')).toHaveText('Hello, Asha!');
```
the page says Hello, Asha

```
screenshot: 'only-on-failure',
```
a picture of the page when a test fails

```
video: 'retain-on-failure',
```
a video of every test, kept only when it fails

```
trace: 'retain-on-failure',
```
a trace of every test, kept only when it fails

TRAP UI mode, the HTML report and the VS Code extension are part of Playwright Test, so they are there for JavaScript and TypeScript. The reporting chapter builds each of them.

TRAP Remove page.pause() before you commit. It needs a headed browser and is only useful while you debug.

6.2 Visual, accessibility and time checks

Screenshot How it looks. Baseline and diff.

Aria snapshot How it reads. Every language.

axe scan Who can use it. WCAG rules.

Fake clock Any date, any delay. No waiting.

Remember Screenshots check pixels, aria snapshots check structure, axe checks access, and the clock checks time. Use each for the mistakes it is good at.

```
await page.addStyleTag({ content: '.card button { background: #b91c1c; }' });
```
a style bug, on purpose

```
.withTags(['wcag2a', 'wcag2aa'])
```
the WCAG 2 A and AA rules

```
await page.clock.runFor('05:00');
```
5 minutes of timers, in a moment

TRAP toHaveScreenshot is part of Playwright Test. Teams add a library for it or use a visual testing service. The aria snapshot below works in every language.

TRAP An automatic scan finds only part of the problems. It cannot tell whether alt text is useful or whether the tab order makes sense. Keep a few manual checks with a keyboard and a screen reader, and use the scan to stop the easy mistakes from coming back.

STEP 7 Framework design

7.1 Page objects and components

Page object One class per screen. Locators in one place.

Component Parts that repeat. Root locator first.

Fixtures Hand them out. No set up in tests.

Checks Stay in the test. Easy to read.

Remember The test says what, the page object says how. A change to the screen is fixed in one class.

```
await loginPage.signIn('asha@shop.example', 's3cret');
```
productsPage always starts signed in

7.2 Framework design, step by step

Settings From the environment. No secrets in Git.

Layers One job each. Data, API, pages.

Fixtures Fresh session per test. Safe in parallel.

Tests By feature, tagged. Steps and checks only.

Remember Framework code and tests live apart. Tests say what, the framework says how, and the environment says where.

```
this.page.once('dialog', dialog => dialog.accept());
```
the shop asks to confirm the order

```
await api.addToCart(session.token, products.waterBottle.id, 2);
```
arrange through the API, test the UI

7.3 CI/CD pipelines

Every push Tests run by themselves. Nobody has to remember.

Fresh machine Install everything each time. Or use the Docker image.

Secrets From the CI settings. Masked in the logs.

Evidence Reports kept, pass or fail. Traces explain failures.

Remember CI runs the same three steps everywhere: a machine that can run browsers, the packages and browsers installed, then the tests with CI settings and secrets.

```
test.only('the cheapest product comes first', async ({ productsPage }) => {
```
left behind after debugging

TRAP Secrets are not given to workflows that run for pull requests from forks, so a fork cannot read your passwords. Tests that need them are skipped or fail there, which is expected.

TRAP Never fix a red pipeline by adding a skip or a longer sleep. Find out whether the app or the test is wrong. The trace from the debugging chapter shows the page at every step of the failed run.

7.4 Reporting labs

For you list, dot, -v. Right now.

For CI Counts and history. Every server reads it.

For testers Error, screenshot, trace. Why it failed.

For everyone Same report, any language. Trends and labels.

Remember Write several reports from one run: a short one for the terminal, JUnit XML for CI, and an HTML report with evidence for people.

TRAP A reporter must not change whether the run passed. It only describes the run, so CI still sees exit code 1 when a test failed.

7.5 What's new, and AI test agents

Releases New browsers each time. Read the notes first.

New APIs In every language. Like consoleMessages.

Test Agents Plan, generate, heal. For Playwright Test.

Playwright MCP A browser for assistants. Any language.

Remember Update on a branch and read the release notes. Let AI write tests, but review them like any other code, and make sure each one can fail.

TRAP The agents write Playwright Test code in TypeScript or JavaScript.